Week 3: The Document Intelligence Refinery

Engineering Agentic Pipelines for Unstructured Document Extraction at Enterprise Scale

1. The Business Objective

The Last Mile of Enterprise Intelligence

In Week 1 you governed code generation. In Week 2 you built a system to judge it. Now you face the problem that blocks every enterprise AI deployment: the data is locked in documents.

Every organization—banks, hospitals, law firms, logistics companies—has its institutional memory trapped in PDFs, scanned reports, slide decks, and spreadsheets. Traditional OCR extracts text but destroys structure. LLMs hallucinate when given raw dumps. The gap between "we have the document" and "we can query it as structured data" costs enterprises billions of dollars annually.

This is not a niche problem. Y Combinator's 2024–2025 batches produced at least eight funded startups (Reducto, Extend, AnyParser, Chunkr, Unsiloed AI, Pulse, Midship, Powder) all attacking this single problem space. The market validation is unambiguous.

The Three Failure Modes You Must Solve

- **Structure Collapse**: Traditional OCR flattens two-column layouts, breaks tables, drops headers. The extracted text is syntactically present but semantically useless.
- **Context Poverty**: Naive chunking for RAG severs logical units. A table split across chunks, a figure separated from its caption, a clause severed from its antecedent—all produce hallucinated answers.
- **Provenance Blindness**: Most pipelines cannot answer "Where exactly in the 400-page report does this number come from?" Without spatial provenance, extracted data cannot be audited or trusted.

The Master Thinker Philosophy

> *An FDE entering a new client engagement has one week before they must demonstrate value. In almost every enterprise context, that value begins with making the client's documents queryable. The FDE who can deploy a document intelligence pipeline in 48 hours—one that actually works on the client's specific document chaos—is irreplaceable.*
>
> *The Master Thinker does not build another PDF-to-text scraper. They build a classification-aware, spatially-indexed, provenance-preserving extraction system that degrades gracefully and knows when to escalate from fast text extraction to expensive vision model analysis.*

2. Your Mission

You will not build a PDF reader. You will act as a Forward Deployed Engineer to architect **The Document Intelligence Refinery**—a production-grade, multi-stage agentic pipeline that ingests a heterogeneous corpus of documents and emits structured, queryable, spatially-indexed knowledge.

Input → Process → Output

INPUT	PIPELINE STAGES	OUTPUT
PDFs (native + scanned) Excel / CSV reports Word docs, slide decks Images with text	1. Document Triage Agent 2. Structure Extraction Layer 3. Semantic Chunking Engine 4. Provenance Index Builder 5. Query Interface Agent	Structured JSON schemas PageIndex navigation tree RAG-ready vector store SQL-queryable fact table Audit trail with page refs

3. Mandatory Research & Conceptual Foundation

You are expected to understand these concepts before writing code. An FDE who cannot explain why vision-based parsing outperforms traditional OCR on financial tables cannot have a credible conversation with a client's data engineering team.

Core Tooling Landscape

- **MinerU (OpenDataLab)** https://github.com/opendatalab/MinerU

  The current state-of-the-art open-source PDF parsing framework. Study its pipeline architecture: PDF-Extract-Kit → Layout Detection → Formula/Table Recognition → Markdown export. Key insight: it uses multiple specialized models, not one general model.

- **Docling (IBM Research)** https://github.com/DS4SD/docling

  Enterprise-grade document understanding with unified DoclingDocument representation. Critical concept: the Document Representation Model—how structure, text, tables, and figures are encoded in a single traversable object. This is the schema you will extend.

- **PageIndex (VectifyAI)** https://github.com/VectifyAI/PageIndex

  A navigation index that gives documents a "table of contents" equivalent for LLM consumption. Study the concept of hierarchical section identification and how it solves the "needle in a haystack" problem for long document RAG.

- **Chunkr (YC S24)** https://github.com/lumina-ai-inc/chunkr

  Open-source API for RAG-optimized chunking. The key innovation: chunk boundaries respect semantic units (paragraphs, table cells, figure captions) rather than token counts. Understand why this matters for retrieval precision.

- **Marker** https://github.com/VikParuchuri/marker

  High-accuracy PDF-to-Markdown converter using layout models. Study how it handles multi-column layouts, equations, and embedded figures—problems that break naive extraction.

Conceptual Foundations

Agentic OCR Pattern

The production pattern: attempt fast text extraction first (pypdf / pdfplumber), measure confidence (character density, whitespace ratio, table completeness), and escalate to vision model only when confidence falls below threshold. Key takeaway: escalation logic is the engineering problem, not the extraction itself.

Spatial Independence & Provenance

Every extracted fact must carry a bounding-box coordinate and page reference. This is the document equivalent of Week 1's content_hash—spatial addressing that remains valid even when content moves. Study how pdfplumber's bbox coordinates work and how to serialize them for audit.

Document-Aware Chunking

Why token-count chunking is wrong for RAG. A 512-token chunk that bisects a financial table produces hallucinations on every query about that table. Study: LlamaIndex's SemanticSplitter, the concept of "logical document units" (LDUs), and hierarchical chunk trees.

VLM vs. OCR Decision Boundary

Vision Language Models (GPT-4o, Gemini Pro Vision, Mistral Pixtral) can "see" document structure but are expensive. Learn the heuristics used in production: scanned vs. digital PDF detection, table detection confidence, handwriting presence. This is a cost-quality tradeoff that every FDE must be able to articulate to a client.

4. The Architecture: The Refinery Pipeline

You will implement a five-stage agentic pipeline. Each stage is a distinct agent with typed input and output schemas. The pipeline is not linear—it has feedback loops, confidence-gated escalation, and a provenance ledger that mirrors Week 1's agent_trace.jsonl.

Stage 1: The Triage Agent (Document Classifier)

Before any extraction begins, every document must be characterized. The Triage Agent produces a DocumentProfile that governs which extraction strategy the downstream stages will use.

Classification Dimensions:

- Origin Type: native_digital | scanned_image | mixed | form_fillable
- Layout Complexity: single_column | multi_column | table_heavy | figure_heavy | mixed
- Language: detected language code + confidence
- Domain Hint: financial | legal | technical | medical | general (used to select extraction prompt strategy)
- Estimated Extraction Cost: fast_text_sufficient | needs_layout_model | needs_vision_model

Implementation: Use pdfplumber character density and whitespace analysis for digital detection. Use a lightweight layout model (or heuristic) for complexity classification. The output is a DocumentProfile Pydantic model stored in .refinery/profiles/{doc_id}.json.

Stage 2: The Structure Extraction Layer (Multi-Strategy)

This is the core engineering challenge. You must implement three extraction strategies and a router that selects the appropriate one based on the DocumentProfile:

- Strategy A — Fast Text (Cost: Low): Tool: pdfplumber or pymupdf. Triggers when: origin_type=native_digital AND layout_complexity IN [single_column]. Confidence gate: the page must have a meaningful character stream (e.g., character count > 100 per page) and image area must not dominate the page (e.g., images < 50% of page area). Define your exact thresholds empirically during Phase 0 exploration and document them in extraction_rules.yaml.
- Strategy B — Layout-Aware (Cost: Medium): Tool: MinerU or Docling. Triggers when: multi_column OR table_heavy OR mixed origin. Extracts: text blocks with bounding boxes, tables as structured JSON, figures with captions, reading order reconstruction.
- Strategy C — Vision-Augmented (Cost: High): Tool: VLM (Gemini Flash / GPT-4o-mini via OpenRouter—budget-aware selection). Triggers when: scanned_image OR Strategy A/B confidence < threshold OR handwriting detected. Pass page images with structured extraction prompts.

Mandatory Pattern — The Escalation Guard: Strategy A must measure extraction confidence before passing output downstream. If confidence is LOW, automatically retry with Strategy B rather than silently passing bad data. This prevents "garbage in, hallucination out" RAG failures.

Stage 3: The Semantic Chunking Engine

Raw extracted content is not RAG-ready. The Chunking Engine converts it into Logical Document Units (LDUs)—semantically coherent, self-contained units that preserve their structural context.

Chunking Rules (your "Constitution" for data quality):

- A table cell is never split from its header row.

- A figure caption is always stored as metadata of its parent figure chunk.
- A numbered list is always kept as a single LDU unless it exceeds max_tokens.
- Section headers are stored as parent metadata on all child chunks within that section.
- Cross-references (e.g., "see Table 3") are resolved and stored as chunk relationships.

Implementation: Define an internal ExtractedDocument Pydantic model that serves as the normalized representation all three extraction strategies must output. This model should capture: text blocks with bounding boxes, tables as structured objects (headers + rows), figures with captions, and reading order. Build a ChunkingEngine class that accepts this ExtractedDocument and emits List[LDU]. Each LDU must carry: content, chunk_type, page_refs, bounding_box, parent_section, token_count, and a content_hash. If you use Docling, your ExtractedDocument can wrap or adapt DoclingDocument. If you use MinerU or a VLM, you must write an adapter that normalizes their output into the same schema.

Stage 4: The PageIndex Builder

Inspired by VectifyAI's PageIndex, this stage builds a hierarchical navigation structure over the document—the equivalent of a "smart table of contents" that an LLM can traverse to locate information without reading the entire document.

The PageIndex is a tree where each node is a Section with: title, page_start, page_end, child_sections, key_entities (extracted named entities), summary (LLM-generated, 2–3 sentences), and data_types_present (tables, figures, equations, etc.).

Critical Use Case: When a user asks "What are the capital expenditure projections for Q3?", the PageIndex allows the retrieval agent to first navigate to the relevant section, then retrieve only the relevant chunks—rather than embedding-searching a 10,000-chunk corpus.

Stage 5: The Query Interface Agent

The front-end of the refinery. A LangGraph agent with three tools: **pageindex_navigate** (tree traversal), **semantic_search** (vector retrieval), and **structured_query** (SQL over extracted fact tables). Every answer must include provenance: the document name, page number, and bounding box of the source.

5. Implementation Curriculum

The following phases are indicatory. The actual challenge requires a working end-to-end system. Your innovation in identifying gaps and engineering solutions beyond these phases is expected and rewarded.

Phase 0: Domain Onboarding — The Document Science Primer

Goal: Understand the problem domain before touching code. An FDE who does not understand the difference between a native PDF's character stream and a scanned PDF's image layer will make the wrong architectural decisions.

1. Read MinerU's architecture documentation end-to-end. Draw the pipeline on paper first.
2. Install pdfplumber and run character density analysis on the provided documents. Observe how character density, bbox distributions, and whitespace ratios differ.
3. Run Docling on the same provided documents. Compare output quality.
4. Deliverable: DOMAIN_NOTES.md documenting: extraction strategy decision tree, failure modes observed, and a hand-drawn (or Mermaid) pipeline diagram.

Phase 1: The Triage Agent & Document Profiling

Goal: Build the classification layer that makes all downstream decisions intelligent.

1. Define DocumentProfile as a Pydantic model with all classification dimensions.
2. Implement origin_type detection: analyze character density, embedded image ratio, and font metadata to distinguish digital vs. scanned.
3. Implement layout_complexity detection using a combination of column count heuristics and table/figure bounding box analysis.
4. Build domain_hint classifier: simple keyword-based approach is acceptable, but implement it as a pluggable strategy so VLM classification can be swapped in.
5. Write unit tests: given a known document type, the profile must classify correctly.

Phase 2: Multi-Strategy Extraction Engine

Goal: Implement the three extraction strategies and the confidence-gated router.

1. Strategy A: Implement a FastTextExtractor wrapping pdfplumber with confidence scoring. Design a multi-signal confidence score using available pdfplumber data: character count, character density (chars / page area in points), image-to-page area ratio, and font metadata presence. A page with high image area but near-zero character count is likely scanned. Define and justify your thresholds in DOMAIN_NOTES.md. Low-confidence pages must trigger escalation.
2. Strategy B: Integrate MinerU or Docling as LayoutExtractor. Implement a DoclingDocumentAdapter that normalizes output to your internal schema.
3. Strategy C: Implement VisionExtractor using a multimodal model via OpenRouter. Build a budget_guard: track token spend per document and log estimated cost. Never let a single document exceed a configurable budget cap.
4. Build ExtractionRouter: a strategy pattern implementation that reads the DocumentProfile and delegates to the correct extractor, with automatic escalation on low confidence.

5. Implement the .refinery/extraction_ledger.jsonl: log every extraction with strategy_used, confidence_score, cost_estimate, and processing_time.

Phase 3: The Semantic Chunking Engine & PageIndex

Goal: Transform raw extraction into RAG-optimized, navigable knowledge.

1. Implement ChunkingEngine with all five chunking rules as enforceable constraints. Build a ChunkValidator that verifies no rule is violated before emitting chunks.
2. Implement content_hash generation for each LDU (same pattern as Week 1's spatial hashing). This enables provenance verification even when document pages shift.
3. Build the PageIndex tree builder: traverse the document's section hierarchy and generate LLM summaries for each section node using a fast, cheap model.
4. Implement the PageIndex query: given a topic string, traverse the tree to return the top-3 most relevant sections before doing vector search. Measure retrieval precision with and without PageIndex traversal.
5. Ingest LDUs into a vector store (ChromaDB or FAISS—local, free tier friendly).

Phase 4: The Query Agent & Provenance Layer

Goal: Build the interface that makes the refinery useful and auditable.

1. Implement the three-tool LangGraph agent: pageindex_navigate, semantic_search, structured_query.
2. Every answer must include a ProvenanceChain: list of source citations with document_name, page_number, bbox, and content_hash.
3. Implement a FactTable extractor: for financial/numerical documents, extract key-value facts (e.g., revenue: $4.2B, date: Q3 2024) into a SQLite table for precise querying.
4. Build the Audit Mode: given a claim ("The report states revenue was $4.2B in Q3"), the system must either verify with a source citation or flag as "not found / unverifiable".

6. The Target Corpus — What You Will Process

Your pipeline must be validated against a heterogeneous corpus that reflects real-world FDE engagement conditions. The corpus is provided as ***data.zip***, containing **50 PDF** documents spanning four document classes. You must successfully process all four document classes (choose from the 50 documents we provided):

- **Class A: Annual Financial Report (PDF, native digital)**
  Commercial Bank of Ethiopia (CBE) Annual Report 2023–24, Fiscal Year Ended June 30, 2024. Challenge: Multi-column layouts, embedded financial tables (income statement, balance sheet), footnotes, cross-references. Source Provided E.g — CBE ANNUAL REPORT 2023-24.pdf
- **Class B: Scanned Government/Legal Document (PDF, image-based)**
  Development Bank of Ethiopia (DBE) Independent Auditor's Report and Financial Statements, 30 June 2023. Challenge: No character stream — pure scanned image. OCR must work correctly to extract any content. Source Provided E.g — Audit Report - 2023.pdf
- **Class C: Technical Assessment Report (PDF, mixed: text + tables + structured findings)**
  Assessment of Financial Transparency and Accountability (FTA) Implementation in Ethiopia — Final Report, August 2022. Submitted by TAK-IRDI to the Ministry of Finance. Challenge: Mixed layout with narrative sections, embedded tables, assessment findings, and hierarchical section structure. Source Provided E.g —fta_performance_survey_final_report_2022.pdf
- **Class D: Structured Data Report (PDF, table-heavy with numerical fiscal data)**
  Ethiopia Import Tax Expenditure Report, FY 2018/19–2020/21, Tax Policy Directorate, Ministry of Finance, September 2022. Challenge: Table extraction fidelity across multi-year fiscal data tables, numerical precision, and structured category hierarchies. Source Provided E.g — tax_expenditure_ethiopia_2021_22.pdf

7. Proof of Execution — The Demo Protocol

To pass, you must submit a video (max 5 minutes) demonstrating the Refinery against a document from the corpus that you did not use during development tuning, or against a document entirely outside the corpus. The demo must follow this exact sequence:

- **Step 1: The Triage**
  Drop a document into the pipeline. Show the DocumentProfile output. Explain your classification decision and which extraction strategy was selected and why.
- **Step 2: The Extraction**
  Show the extraction output side-by-side with the original document. Point to one table and show it was extracted as structured JSON with correct headers and values. Show the extraction_ledger.jsonl entry with confidence score.
- **Step 3: The PageIndex**
  Show the PageIndex tree for the document. Navigate it to locate a specific piece of information without using vector search.
- **Step 4: The Query with Provenance**
  Ask the system a natural language question about the document. Show the answer AND its ProvenanceChain (page number + bounding box citation). Then open the original PDF to the cited page and verify the claim.

8. Deliverables

Interim Submission -- Thursday 03:00 UTC

- Report (a SINGLE PDF containing):

1. **Domain Notes (Phase 0 deliverable)**

    - Extraction strategy decision tree
    - Failure modes observed across document types
    - Pipeline diagram (Mermaid or hand-drawn)

2. **Architecture Diagram**

    - Full 5-stage pipeline with strategy routing logic

3. **Cost Analysis**

    - Estimated cost per document for each strategy tier (Strategy A / B / C)

- GitHub Repository:

Core Models

- `src/models/` -- All Pydantic schemas fully defined: `DocumentProfile`, `ExtractedDocument`, `LDU`, `PageIndex`, `ProvenanceChain`

Agents & Strategies (Phases 1-2)

- `src/agents/triage.py` -- Working Triage Agent: origin_type detection, layout_complexity detection, domain_hint classifier
- `src/strategies/` -- All three extraction strategies with shared interface: `FastTextExtractor`, `LayoutExtractor`, `VisionExtractor`
- `src/agents/extractor.py` -- `ExtractionRouter` with confidence-gated escalation guard

Configuration & Artifacts

- `rubric/extraction_rules.yaml` -- Externalized chunking constitution and extraction thresholds
- `.refinery/profiles/` -- `DocumentProfile` JSON outputs for at least 12 corpus documents (minimum 3 per class)
- `.refinery/extraction_ledger.jsonl` -- Ledger entries with strategy selection, confidence scores, and cost estimates for the same documents

Project Setup

- `pyproject.toml` with locked dependencies
- `README.md` with setup and run instructions

Tests

- Unit tests for Triage Agent classification and extraction confidence scoring

---

Final Submission -- Sunday 03:00 UTC

- Report (a SINGLE PDF containing):

1. **Everything from interim, refined**
2. **Extraction Quality Analysis**
    - Precision/recall on table extraction across the corpus
3. **Lessons Learned**
    - At least two cases where the initial approach failed and how it was fixed

- GitHub Repository:

Everything from interim, plus:

Agents (Phases 3-4)

- `src/agents/chunker.py` -- Semantic Chunking Engine with all 5 chunking rules enforced via `ChunkValidator`
- `src/agents/indexer.py` -- PageIndex tree builder with LLM-generated section summaries
- `src/agents/query_agent.py` -- LangGraph agent with 3 tools: `pageindex_navigate`, `semantic_search`, `structured_query`

Data Layer

- FactTable extractor with SQLite backend for numerical documents
- Vector store ingestion (ChromaDB or FAISS) of all LDUs
- Audit Mode: claim verification with source citation or "unverifiable" flag

Artifacts

- `.refinery/pageindex/` -- PageIndex trees (JSON) for at least 12 corpus documents (minimum 3 per class)
- 3 example Q&A per document class (12 total), each with full `ProvenanceChain` citations, using documents from different classes

Infrastructure

- `Dockerfile` (recommended)

- Video Demo (max 5 minutes):

Following the Demo Protocol sequence:

1. **Triage** -- Drop a document, show `DocumentProfile`, explain strategy selection
2. **Extraction** -- Side-by-side with original, structured JSON table output, ledger entry with confidence score
3. **PageIndex** -- Tree navigation to locate specific information without vector search

4. **Query with Provenance** -- Natural language question, answer with `ProvenanceChain`, verify against source PDF

9. Evaluation Rubric

Metric	Score 1 — The Vibe Coder	Score 3 — Competent Engineer	Score 5 — Master Thinker
Extraction Fidelity	Single strategy. Naive text dump. Tables are broken strings. No confidence measurement.	Multi-strategy implemented. Tables extracted as JSON. Basic confidence scoring.	Escalation guard working. Table extraction verified with ground truth. Confidence-gated routing demonstrated. Scanned doc handled correctly via VLM.
Architecture Quality	Single script. Hardcoded paths. No Pydantic schemas. Strategies intermingled.	Separate modules per stage. Pydantic models exist. Basic error handling.	Clean strategy pattern. Fully typed pipeline. Each stage is independently testable. Cost budget guard implemented.
Provenance & Indexing	No page citations. No PageIndex. Cannot verify claims against source.	Page numbers included in answers. Basic section index built.	Full ProvenanceChain with bbox coords and content_hash. PageIndex traversal demonstrated to outperform naive vector search on section-specific queries.
Domain Onboarding	No DOMAIN_NOTES. No evidence of understanding document science. Architecture is copy-paste.	DOMAIN_NOTES present. Shows understanding of OCR vs. digital PDF distinction.	Deep analysis of failure modes across document classes. Can articulate the VLM cost tradeoff precisely. Architecture decisions are grounded in domain understanding, not just "it worked."
FDE Readiness	Pipeline only works on provided test docs. Not deployable to new corpus without code changes.	Pipeline runs on any PDF. Configuration is partially externalized.	A new document type can be onboarded by modifying only extraction_rules.yaml, not code. The system degrades gracefully on unseen layouts. README enables deployment in under 10 minutes.

The FDE Insight: The ability to onboard to a new document domain in 24 hours—understanding its structure, its failure modes, and the correct extraction strategy—is precisely what separates a forward-deployed engineer from a developer who can only work in familiar territory.